Assignment 4: CNN Accelerator

Due Date

- **Report & Code Submission – June 19th (Fri), 11:59 PM**
- **Design Review – May 20th, 22nd**
- **Final Presentation – June 5th**

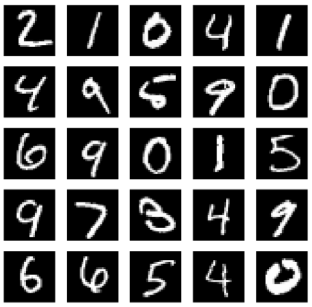
Notice

- **Design a *CNN accelerator IP* module using Verilog. Additionally, create a *programming system (block design)* to control the module, and develop the corresponding *control code written in C (Vitis)*.**
- Please submit the report individually, and only one person per team should submit the design project.
- Compress your Vivado project folder and driver codes into a .zip file for submission. The compressed folder should include not only the top design but also all project folders for the sub-IP blocks. Additionally, make sure that design source files such as .v and .xdc are also included.

The goal of this project is to design a CNN accelerator that classifies 10,000 images from the MNIST dataset using the provided CNN weight parameters, with a focus on optimizing power consumption, resource utilization, and latency. Additional credit will be awarded for efforts to design optimized hardware that utilizes even lower bit precision than the given parameters. It is recommended to develop an optimal hardware architecture that meets the specified requirements. All computations and data alignment required for CNN acceleration must be performed within the PL (Programmable Logic). The PS (Processing System) may only be used to sequentially write image data to memory accessible by the PL, monitor the status of the PL, and read back the results. For further details, please refer to the main guidelines below.

MNIST Dataset

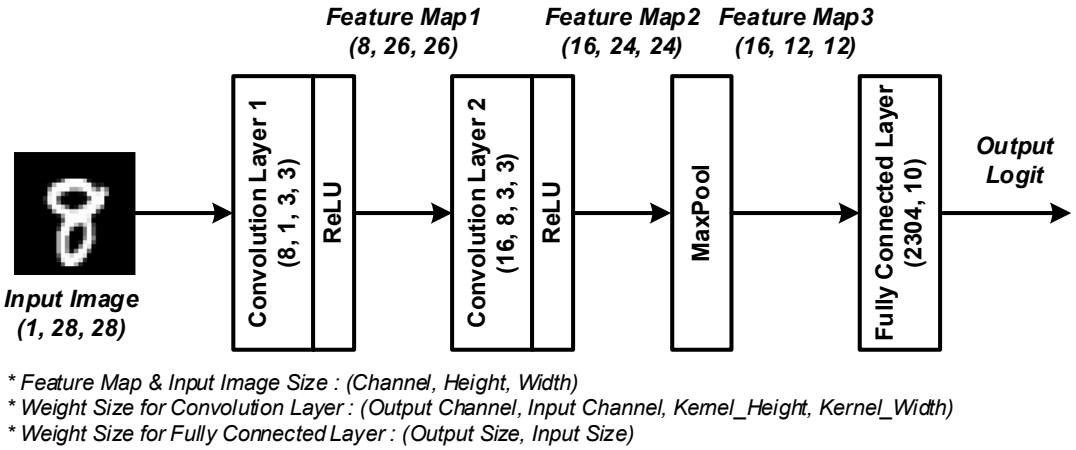
The MNIST dataset is a representative image dataset consisting of handwritten digits (from 0 to 9). It is one of the most widely used benchmarks in machine learning, especially in the fields of image classification and deep learning. Each image is a grayscale image with a size of 28×28 pixels, and each pixel is represented as an 8-bit unsigned integer with a value ranging from 0 to 255. The MNIST dataset provides a simple yet effective environment for experimenting with handwritten digit recognition problems, making it frequently used for comparing the performance of various algorithms or testing new neural network architectures.



< Fig. 1 Example of 2D Convolution >

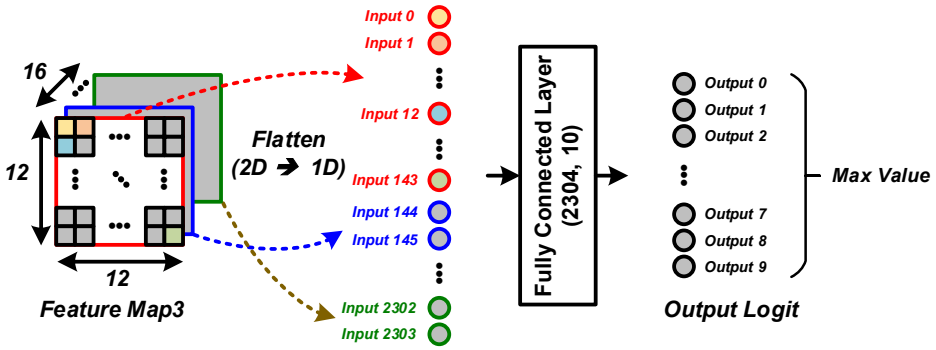
CNN Architecture

Fig.2 shows the CNN targeted for acceleration in this project, which consists of two convolution layers, one MaxPool layer, and one fully connected layer. Each convolution layer uses a 3×3 kernel with a stride of 1, and zero padding is not applied, so the width and height of the output feature map decrease by 2 compared to the input. The first convolution layer has 8 output channels, and the second layer has 16 output channels. A ReLU function is applied after each convolution layer. The MaxPool layer uses a 2×2 filter, resulting in an output of (16, 12, 12) in the order of (channel, height, width).



< Fig. 2 Target CNN Network Architecture >

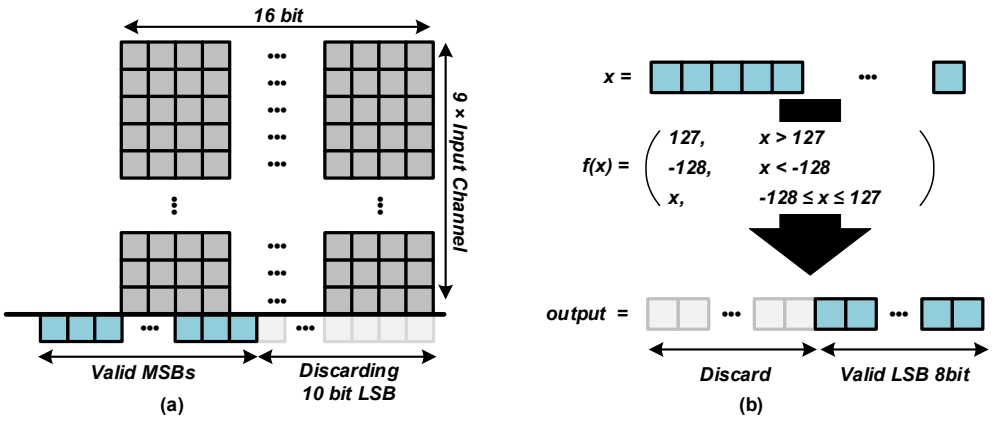
To use the output of the MaxPool layer as the input to the fully connected layer, it must be converted into a 1D vector. In this process, the data is arranged contiguously in the order of width, height, and channel as show in Fig. 3. As a result, the input to the fully connected layer is a 1D vector with 2,304 dimensions. The fully connected layer outputs a 1D vector with 10 dimensions (logit values for each class), and the index of the largest value among these is selected as the final result (predicted digit).



< Fig. 3 Example of Data Alignment for Fully Connected Layer and Output Logit >

Bit Precision

The target network in this project is trained using quantization to signed 8-bit integers. The input image data is preprocessed as signed 8-bit integers, converted from the original unsigned 8-bit integers (all input data is provided as .npy files). During inference, all feature maps must be transformed to signed 8-bit integers immediately after each convolution or fully connected operation, as illustrated in Fig. 4. During layer operations, bit extension is required to minimize error in repeated addition and multiplication processes. When truncating the bit-extended data back to signed 8-bit integers, the following method is used: first, data excluding the least significant 10 bits (LSB 10 bits) is truncated. If the resulting value exceeds +127, it is saturated to 127; if it is less than -128, it is saturated to -128. Finally, only the least significant 8 bits are used as the output.



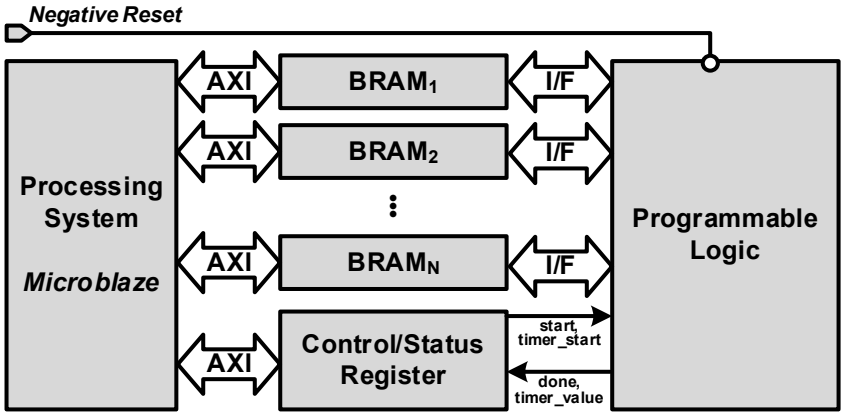
< Fig. 4 Bit Truncation and Saturation for 8-bit Quantization >

When carrying out the assignment, you should first design an accelerator that supports the provided signed 8-bit integer weights and bit precision. Additional credit will be awarded if you also implement lower-bit quantization and design a dedicated architecture. In both the design review and the final presentation, you must specify whether you have implemented lower-bit quantization and design a dedicated architecture, providing details about your implementation. In this case, both the accelerator based on signed 8-bit integers and the additionally designed accelerator must be verified and submitted, and performance improvements must be reported in your presentation.

Design Constraints

The objective of this assignment is to design a CNN accelerator with optimized hardware architecture. Therefore, performing CNN acceleration operations using the PS (Processing System) is strictly prohibited. The PS must only perform the following tasks: reading from and writing to memory connected to the PL (Programmable Logic) in address order, and providing a start signal to the PL or checking the done signal via control/status registers. Any other operations, such as rearranging data for convolution or fully connected computations, or performing specific operations like MaxPool or ReLU on the PS side, will not be given any points. Additionally, it is prohibited to convert image or weight data into .coe format and initialize BRAM with it. All initial image and weight data must be transferred from the PS.

As shown in Fig. 5, the overall system configuration is similar to Assignment 3, but the internal BRAM structure can be freely designed. You may also define and use the Control/Status Registers according to your own design. However, you must write the .c code to control these registers yourself, using the week 9 course materials as a reference. For debugging and testing convenience, please configure the reset signal for the PL separately using an external switch in this assignment as well.



< Fig. 5 System block diagram for the target design >

The latency of the accelerator is defined as the time from the moment data is first written to the BRAM to the moment when all outputs processed by the PL are read back by the PS. For example, suppose computations are performed sequentially for 10,000 images as shown in Fig. 6. The PS should start timing just before the weight parameters are sent. After all weight parameters have been transferred, the PS repeatedly performs the following steps in a loop: sending the input image, issuing the start command, checking the done state, and reading the output. Timing should stop when the output read for the last image is completed. Timing logic must be implemented in the PL. The timer shall count from the first weight write to the last output read, and expose the elapsed time in milliseconds through a CSR register for the PS to read and report. The PS triggers the PL timer by writing to the CSR_TIMER_START register before the first weight transfer begins. The figure below provides an example of measuring latency using the time library; however, you are not required to control the PS exactly as shown in the example.

```c
// CSR register offsets
#define CSR_DONE         0x00U   // [0] accelerator done flag
#define CSR_START        0x04U   // [0] accelerator start signal
#define CSR_TIMER_START  0x08U   // [0] timer start/stop
#define CSR_TIMER_VALUE  0x10U   // elapsed ms

// Basic parameters
#define WEIGHT_LEN  24263U
#define IMAGE_LEN     784U
#define OUTPUT_LEN     10U
#define NUM_IMAGES  10000U

int main(void)
{
    int8_t result[OUTPUT_LEN];

    init_platform();
    Xil_DCacheDisable();

    // 1. Start timer
    Xil_Out32(CSR_BASE + CSR_TIMER_START, 1U);

    // 2. Send weight parameters to WMEM
    for (u32 i = 0U; i < WEIGHT_LEN; i++)
        Xil_Out32(WMEM_BASE + i * 4U, (u32)(int8_t)weight_data[i]);

    for (u32 i = 0U; i < NUM_IMAGES; i++) {
        // 3. Send input image to IMEM
        for (u32 j = 0U; j < IMAGE_LEN; j++)
            Xil_Out32(IMEM_BASE + j * 4U, (u32)(int8_t)input_data[i * IMAGE_LEN + j]);

        // 4. Start accelerator
        Xil_Out32(CSR_BASE + CSR_START, 1U);
        for (volatile int d = 0; d < 10; d++);
        Xil_Out32(CSR_BASE + CSR_START, 0U);

        // 5. Wait for Done
        while ((Xil_In32(CSR_BASE + CSR_DONE) & 1U) == 0U);

        // 6. Read output logits from OMEM
        for (u32 k = 0U; k < OUTPUT_LEN; k++)
            result[k] = (int8_t)Xil_In32(OMEM_BASE + k * 4U);
    }

    // 7. Stop timer
    Xil_Out32(CSR_BASE + CSR_TIMER_START, 0U);

    // 8. Read elapsed time
    u32 elapsed_ms = Xil_In32(CSR_BASE + CSR_TIMER_VALUE);
    xil_printf("Runtime: %ums\r\n", (unsigned)elapsed_ms);

    cleanup_platform();
    return 0;
}
```

< Fig. 6 Examples of PS control code and latency measurement >

**Note that this figure is only for example*

Parameters

The provided .zip includes five .npy files. These files contain: the weights for the three layers that make up the CNN, 10,000 MNIST images, and the corresponding CNN output logits for those images. Since MicroBlaze cannot directly read .npy or image files at runtime, and using UART for data transfer would cause communication delay to dominate the total measured latency, we strongly recommend converting all .npy data into C header files in advance. When the project is built and run, these arrays will be loaded into MicroBlaze's DRAM automatically, and your driver code can then transfer the data to the PL memory regions (IMEM, WMEM) via AXI.

Follow the steps below.

Step 1. Convert .npy files to C header files using the provided Python script in your python environment. Refer to Fig. 7 for an example that converts input.npy and the weight .npy files into .h. The given example flattened the parameters without reordering, but you may be required to reorder the weight axes to optimize with your hardware.

```python
import numpy as np

# Load input.npy : dtype=int8, shape=(10000, 1, 28, 28)
data = np.load("input.npy")
print(f"input.npy data type: {data.dtype}")
print(f"input.npy shape: {data.shape}")

flat = data.flatten()           # (10000 * 784,) int8
N    = len(flat)                # 7,840,000

with open("input_data.h", "w") as f:
    f.write("#ifndef INPUT_DATA_H\n")
    f.write("#define INPUT_DATA_H\n\n")
    f.write(f"#define NUM_IMAGES  10000U\n")
    f.write(f"#define IMAGE_LEN     784U   /* 1*28*28 */\n\n")
    f.write("static const int8_t input_data[10000 * 784] = {\n")
    for i in range(0, N, 16):
        chunk = flat[i : i + 16]
        hex_vals = ", ".join(f"0x{int(v) & 0xFF:02X}" for v in chunk)
        f.write(f"    {hex_vals},\n")
    f.write("};\n\n")
    f.write("#endif /* INPUT_DATA_H */\n")

print(f"Done: input_data.h ({N} values)")
```

```
✓ 2.8s
input.npy data type: int8
input.npy shape: (10000, 1, 28, 28)
Done: input_data.h (7840000 values)
```

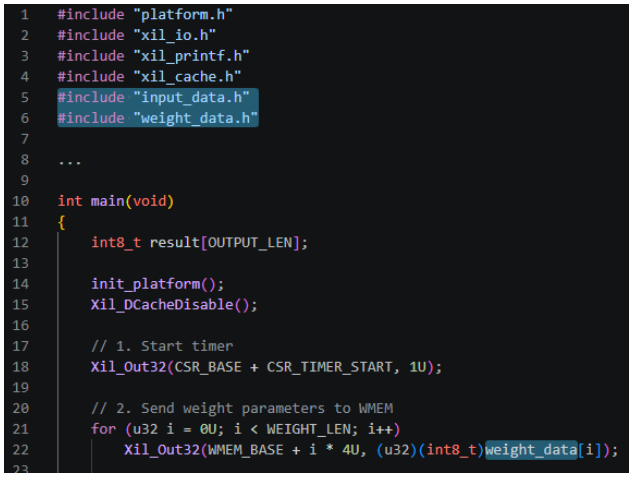
```c
// input_data.h > ...
#ifndef INPUT_DATA_H
#define INPUT_DATA_H

#define NUM_IMAGES  10000U
#define IMAGE_LEN     784U   /* 1*28*28 */

static const int8_t input_data[10000 * 784] = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x0
    0x53, 0x7F, 0x7B, 0x3F, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
    0x0F, 0x12, 0x2F, 0x4D, 0x55, 0x7E, 0x7E, 0x7E, 0x7E, 0x7
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
```

< Fig. 7 Example of converting numpy array into c header file using Python >

**Note that this figure is only for example*

Step 2. Place the generated .h files in the same source directory as your main.c, or add the directory to the include path in your Vitis project settings. Then include the generated header files in your Vitis

project and use them as shown in the example below.



```c
#include "platform.h"
#include "xil_io.h"
#include "xil_printf.h"
#include "xil_cache.h"
#include "input_data.h"
#include "weight_data.h"

...

int main(void)
{
    int8_t result[OUTPUT_LEN];

    init_platform();
    Xil_DCacheDisable();

    // 1. Start timer
    Xil_Out32(CSR_BASE + CSR_TIMER_START, 1U);

    // 2. Send weight parameters to WMEM
    for (u32 i = 0U; i < WEIGHT_LEN; i++)
        Xil_Out32(WMEM_BASE + i * 4U, (u32)(int8_t)weight_data[i]);
```

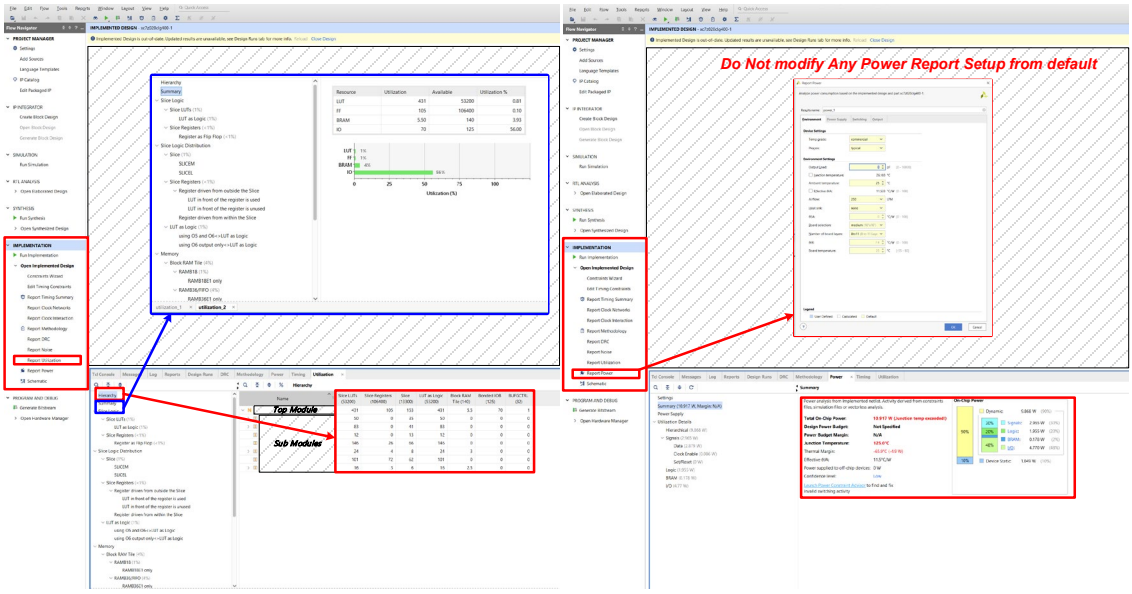
< Fig. 8 Example including data header files to hardware driver >
**Note that this figure is only for example*

Tips

After implementing your system in the VIVADO tool, you will receive reports on both the utilization of FPGA resources and the estimated power consumption of your design when implemented on the board. Refer to Fig. 9 for details. You are encouraged to check the utilization and power reports for your designed system and make efforts to optimize them.

The priorities for this project are as follows:

1. ***Maximize the utilization*** of the available FPGA resources to achieve the ***shortest possible latency***.
2. Design for ***minimal power consumption as long as it does not result in reduced throughput*** (or increased latency).
3. ***Every design must be accompanied by a logical explanation of your design choices***, and you should be able to justify them during the design review and final presentation.
4. The final grade for the project will be determined by a comprehensive evaluation, not just by simple quantitative metrics.



< Fig. 9 Examples of implementation report in VIVADO >

First, it is strongly recommended that you begin by implementing a module that can perform CNN computation for a single image. This module will serve as your baseline. You should then try applying various techniques covered in class to improve performance beyond this baseline. Be sure to compare the results and present these comparative metrics during your design review and final presentation.